

RTO Guard

Autonomous AI Agent System for Cash-on-Delivery (COD) Return-to-Origin Reduction in Indian D2C Brands

AUTHOR / CANDIDATE

Anshul Mathur

CORE TECH STACK

LightGBM · LangGraph ·
pgvector · Gemini

GITHUB REPOSITORY

anshul-1107/RTO_Guard_AI

NET SAVINGS IMPACT

₹48,000 / 20k orders (~9x
Naive Baseline)

9x

More net savings than naive
"verify all COD" rules

0%

PII leakage to LLM
(Automated SHA
tokenization)

100%

Guaranteed rule enforcement
(LLM cannot violate caps)

43 / 43

Automated test cases
passing in CI pipeline

1. Executive Summary & Problem Space

In Indian e-commerce and direct-to-consumer (D2C) brands, **Cash on Delivery (COD)** represents **60%–70% of total order volume**. However, COD suffers from an alarming return-to-origin (RTO) rate of **25% to 35%**, compared to just **~5% for prepaid orders**.

The Real Unit Economics of an RTO:

When an order is returned to origin, the brand incurs direct deadweight financial loss across multiple cost buckets:

Cost Component	Average INR Loss	Operational Explanation
Forward Logistics	₹60 - ₹80	Courier charges to dispatch parcel from central warehouse to customer hub.
Reverse Logistics	₹60 - ₹90	Return courier charges when parcel is rejected, unreachable, or cancelled at doorstep.
Packaging & Damage	₹30 - ₹50	Torn packaging, lost polybags, and products returned in unsellable condition.
Inventory Blockage	₹30 - ₹60	Stock is locked in transit for 7–14 days, resulting in stockouts for paying customers.
Total Loss per RTO	₹180 - ₹280	Direct EBITDA reduction for every single failed delivery!

Why Naive Industry Approaches Fail:

- **Naive Rule 1 ("Call every COD customer"):** Verification calls cost ₹8–₹12 per call. Calling 100% of COD orders generates massive operational friction, annoys genuine customers, and yields only ₹5,000 net savings per 20,000 orders.
- **Naive Rule 2 ("Disable COD completely"):** Causes 40%–60% of genuine buyers to abandon checkout and purchase from competitors, crippling top-line revenue.
- **RTO Guard Solution:** A dual-layer system combining **Calibrated Machine Learning (LightGBM)** for economic risk thresholding with an **Autonomous LLM Agent (LangGraph + RAG + Guardrails)** to orchestrate polite, context-aware customer interventions.

2. End-to-End System Architecture



Core Components & Directory Breakdown:

Module / Directory	Key Responsibility	Tech Stack & Key Files
agent/	State graph, decision logic, prompt templates, and PII anonymization.	LangGraph, Pydantic, graph.py , policy.py , pii.py
mL/	Feature engineering, model training, calibrated inference, and SHAP explainability.	LightGBM, Scikit-learn, train.py , predict.py , economics.py
rag/	Chunking, embedding, vector store upsert, and hybrid keyword/vector search.	PostgreSQL + pgvector, Gemini Embeddings / FakeEmbedder, retriever.py

Module / Directory	Key Responsibility	Tech Stack & Key Files
dashboard/	Interactive operational console for warehouse teams, ops approval, and evals.	Streamlit, Altair, app.py , data.py
evals/	Continuous quality gates: retrieval MRR, LLM-as-a-judge, and agent policy adherence.	Pytest, Golden Datasets, eval_agent.py , eval_retrieval.py

3. Deep-Dive: Key Technical Innovations

A. Economic Thresholding vs. Generic Accuracy

Standard ML pipelines optimize for accuracy, F1-score, or ROC-AUC. In production RTO reduction, **maximizing net financial savings** is the only objective that matters. We define an economics matrix in `ml/economics.py`:

```
Net Value = Flagged * [ (is_rto * P_success * RTO_Cost) - ((1 - is_rto) * P_friction * Order_Margin) - Interv
Where:
- RTO_Cost = ₹180 (forward + reverse logistics + packaging)
- Intervention_Cost = ₹10 (WhatsApp API + agent verification call)
- P_success = 0.40 (40% of flagged risky orders are saved)
- P_friction = 0.05 (5% of genuine customers cancel due to nuisance)
- Order_Margin = ₹200 (profit lost if genuine customer cancels)
```

The LightGBM threshold is chosen on the validation set to maximize this exact function, resulting in an optimal threshold of **0.22** instead of the arbitrary 0.50. This captures 64% of all RTOs while flagging only 33% of orders.

B. Zero PII Guardrail (DPDP Act & GDPR Compliance)

External LLM APIs (OpenAI, Gemini, Anthropic) should never receive sensitive customer PII. In `agent/pii.py`:

- **Phone numbers:** Masked to `98XXXXXX12`.
- **Customer Names:** Replaced with template variable `{first_name}`; the real name is injected locally in memory right before WhatsApp dispatch.
- **Full Addresses:** Never sent to the LLM. Replaced by structural feature flags: `has_house_no: bool`, `has_landmark: bool`, `address_len: int`.

C. "LLM Proposes, Rules Enforce" Architecture

LLMs are probabilistic and prone to hallucinations or adversarial prompting. RTO Guard implements a deterministic enforcement boundary in `agent/policy.py`:

Guaranteed Invariants Enforced by Code:

- Prepaid discount can **never exceed 10% or ₹150** (enhanced cap ₹250 requires human approval).
- Total discount on an item cannot exceed **50%**.
- Customers with **≥ 2 prior RTOs** can never be auto-shipped on COD without a ₹99 token advance.
- Orders with `ACTION_CANCEL` or `OPS_HOLD` **must pause for human approval**.
- Customer-facing messages are regex-filtered against banned internal terms: `["fraud", "suspicious", "risk", "blacklist", "rto", "score"]`.

D. Durable Human-in-the-Loop with LangGraph Interrupts

When an order requires ops approval (e.g., high-value COD order or order hold), LangGraph executes `interrupt()`. The state is serialized and persisted into PostgreSQL via `PostgresSaver`. If the application restarts, crashes, or is redeployed, the workflow does not restart:

```
# Inside agent/graph.py
if decision.requires_approval and not state.get("approved"):
    user_decision = interrupt({
        "order_id": order["order_id"],
        "proposed_action": decision.action,
        "reasons": decision.reasons
    })
# Execution pauses here until Ops approves on Streamlit Dashboard!
```

4. Production Engineering & Deployment Blueprint

High-Throughput Microservice Architecture

Layer	Production Implementation	Scale & SLA Target
Ingestion	Shopify / Magento Webhook → AWS API Gateway → AWS SQS / Kafka queue	Handles 1,000+ orders/sec spike during flash sales
ML Inference	LightGBM exported to ONNX / C++ runtime served on AWS ECS Fargate	p99 Latency < 15ms per order
Stateful Agent	LangGraph Workers running on Kubernetes (EKS) with horizontal pod autoscaler	Processes decision in < 800ms; durable checkpoint in Supabase / RDS
Vector DB	Managed AWS RDS PostgreSQL 16 with pgvector & HNSW indexing	Retrieval latency < 30ms for top-5 chunks
Communication	WhatsApp Business API (Gupshup / Karix / Meta Cloud API) + Interactive CTA buttons	Asynchronous delivery; webhook callbacks for button clicks
Observability	Langfuse (Token cost, latency, LLM traces) + Prometheus & Grafana (System metrics)	Real-time tracking of cost-per-order and LLM violation rate

5. The Ultimate Interview Masterclass (Q&A)

PITCH Q1: Can you give a 60-second elevator pitch of RTO Guard?

"RTO Guard is an AI agent system designed for Indian D2C brands to tackle the 30% Return-to-Origin rate on Cash-on-Delivery orders, which burns ₹180 to ₹250 per returned parcel. Instead of blunt rules like 'call everyone' or 'cancel COD', RTO Guard combines a LightGBM ML model calibrated for net rupee savings with a LangGraph LLM agent. The ML model predicts risk and explains drivers with TreeSHAP. The agent retrieves brand policies via hybrid pgvector RAG, decides a targeted intervention (like a ₹99 token advance or 5% prepaid discount), enforces deterministic guardrails, and drafts personalized WhatsApp messages, keeping human-in-the-loop for costly decisions. It achieves ~9x higher net savings than baseline rules while ensuring zero PII reaches the LLM."

DESIGN Q2: Why combine traditional ML (LightGBM) with an LLM Agent (LangGraph)? Why not use just one?

"Each technology solves what the other is bad at:

- **Why not LLM only?** LLMs cannot perform statistical tabular probability calibration across 100,000 historical records. They are slow (500ms–2s), expensive (~\$0.01 per order), and prone to numeric hallucinations.
- **Why not ML only?** ML outputs a cold scalar probability (e.g. 0.42). It cannot interpret nuanced brand SOPs, understand unstructured address issues, draft persuasive customer messages in regional Hinglish, or navigate complex stateful negotiation flows.
- **The Synergy:** LightGBM does the fast, cheap (0.5ms) math and provides TreeSHAP explanations. LangGraph uses those explanations as structured context to consult policy SOPs and communicate with the customer."

ML ECONOMICS Q3: Why did you not use `scale_pos_weight` or optimize for standard ROC-AUC / F1-score?

"In financial decision systems, probability calibration is critical. Techniques like `scale_pos_weight` distort the output probabilities away from true posterior likelihoods. Because our agent calculates real INR trade-offs (savings from prevented RTOs vs. cost of interventions and friction loss from genuine customer churn), we kept raw probabilities calibrated. We then used grid search on the validation set against our business utility function (`ml/economics.py`) to find the threshold that delivers maximum INR savings. An optimal threshold of 0.22 yielded ₹48,000 savings per 20,000 orders."

RAG Q4: Why was Hybrid Search (pgvector + BM25) chosen over pure semantic vector search?

"Pure vector search frequently struggles with exact domain tokens, policy acronyms, and numeric thresholds (e.g., '₹99 advance', 'Tier-3', 'NDR', '5% cap', 'Bluedart SLA'). BM25 provides exact keyword and token matching, while pgvector provides semantic conceptual matching (e.g., 'customer refused delivery multiple times' matches 'Repeat RTO Policy'). We combined both using Reciprocal Rank Fusion (RRF), ensuring top chunks always contain both exact rule numbers and conceptual relevance."

SAFETY Q5: How do you guarantee the LLM doesn't offer a 50% discount or hallucinate policies?

"We follow the 'LLM Proposes, Code Enforces' design pattern. The LLM generates a structured JSON output constrained by Pydantic schemas. That output immediately passes into `agent/policy.py`, a pure Python deterministic validation layer. If the LLM proposes an 8% discount on a product where brand policy allows at most 5%, the code clamps the value to 5% and records a policy violation metric. The LLM is never given direct write access to customer accounts or discount generation APIs."

HITL Q6: How does Human-in-the-Loop work across server restarts and distributed workers?

"We leveraged LangGraph's native `interrupt()` pattern backed by `PostgresSaver` checkpointing. When a high-risk order requires ops approval, the graph pauses and persists the entire execution state (thread ID, order facts, LLM proposal) into PostgreSQL. The workflow thread becomes dormant. When an operations executive reviews and clicks 'Approve' or 'Reject' on the Streamlit dashboard hours later, the dashboard issues `agent.invoke(Command(resume=decision), config)`. The worker rehydrates the state from PostgreSQL and continues execution right from the next node."

SCALE Q7: If order volume spikes from 100k to 10M orders/month, what would you re-architect?

"1) **Decouple scoring from agent execution:** LightGBM runs synchronously at checkout/order-create via an in-memory C++ / ONNX service in <10ms. Low-risk orders (67%) auto-ship immediately without touching queues or LLMs.
2) **Asynchronous SQS/Kafka Queue for Agent:** Risky orders (33%) push to a partitioned Kafka topic to level-load agent invocations.
3) **Embedding & LLM Caching:** Cache frequent policy retrieval vectors in Redis. Cache deterministic message templates for common reason-triplets so the LLM is only invoked for edge-case reasoning.
4) **Database Read Replicas & HNSW:** Migrate pgvector to dedicated read-replicas with HNSW indexing to support thousands of concurrent vector queries."

CI/CD Q8: How do you prevent agent regressions in automated CI/CD pipelines?

"In `evals/`, we have two automated quality gates run on every GitHub Pull Request:

- **Retrieval Evaluation (`eval_retrieval.py`):** Tests 24 golden queries against pgvector, ensuring Mean Reciprocal Rank (MRR) > 0.85 and Recall@3 > 0.90.
- **Agent Decision Evaluation (`eval_agent.py`):** Runs 10 multi-edge test cases (repeat RTOs, high-value festive orders, incomplete addresses) and asserts that guardrails catch 100% of illegal proposals, and that LLM-as-a-judge scores message empathy and clarity > 4.0/5.0."